

Hackermannn!!!

A curious student from the Department of Computer Science and Engineering (CSE) at Shanto-Mariam University of Creative Technology (SMUCT) is determined to obtain the hidden problem set for the upcoming SMUCT Intra University Programming Contest. To do so, the student secretly enters the contest coordinator's office and finds a notebook containing n possible access codes. Each access code is a non-empty string consisting of lowercase English letters.

After examining the contest system, the student discovers that the system stores exactly one of the access codes from the notebook as the administrator password. The authentication system considers two access codes equivalent according to the following rules:

- Two access codes A and B are directly equivalent if they share at least one common letter.
- Two access codes A and B are also considered equivalent if there exists another access code C from the notebook such that A is equivalent to C , and C is equivalent to B .

If the actual administrator password is an access code, then entering any equivalent access code will successfully unlock the contest system.

For example, suppose the notebook contains the following access codes:

a
b
ab
d

Then the access codes "a", "b", and "ab" are all equivalent because they are connected through common letters, while "d" forms a separate equivalence group.

- If the administrator password is "b", the student can gain access using "a", "b", or "ab".

- If the administrator password is "d", only "d" will work.

Since the student does not know which access code is actually being used by the contest system, they want to determine the minimum number of access codes they must try to guarantee gaining access to the hidden problem set.

Input Format

- The first line contains an integer n ($1 \leq n \leq 2 \times 10^5$) — the number of access codes in the notebook.
- The next n lines each contain a non-empty string s_i consisting of lowercase English letters, representing an access code.

Constraints

- Each access code has length at most 50.
- Some access codes may be identical.
- The total length of all access codes does not exceed 10^6 characters.

Output Format

Print a single integer — the minimum number of access codes the SMUCT student must try to guarantee access to the contest system, regardless of which access code is the administrator password.

Thesis Marketing

The Department of Computer Science and Engineering (CSE) is assigning undergraduate students to thesis advisors. There are n students, numbered from 1 to n .

Each student either:

- has no thesis advisor (they are the principal researcher leading an independent project), or
- has exactly one direct thesis advisor, who is another researcher with a different ID.

A researcher A is considered the academic advisor of another researcher B if at least one of the following is true:

- Researcher A is the direct thesis advisor of researcher B.
- Researcher B has a direct thesis advisor C, and researcher A is the academic advisor of C.

It is guaranteed that the advising structure contains no cycles. In other words, no researcher can directly or indirectly become the advisor of their own advisor.

At the end of the semester, the department is organizing a Research Progress Review. All researchers must be divided into the minimum number of presentation sessions, where each researcher is assigned to exactly one session.

To avoid conflicts of interest during evaluation, no presentation session may contain both a thesis advisor and any of their direct or indirect advisees.

Determine the minimum number of presentation sessions required.

Input Format

- The first line contains an integer n — the number of researchers.
- The next n lines each contain an integer p_i .
 - If $p_i = -1$, researcher i has no thesis advisor.
 - Otherwise, p_i is the ID of the direct thesis advisor of researcher i .

It is guaranteed that:

- No researcher is their own direct thesis advisor.
- The advising hierarchy contains no cycles.

Constraints

- $1 \leq n \leq 2000$
- $1 \leq p_i \leq n$ or $p_i = -1$

Output Format

Print a single integer — the minimum number of presentation sessions required.

Mission: Welcome

A programming contest is being held in the Department of Computer Science and Engineering at Shanto Mariam University of Creative Technology (SMUCT). As a participant, your first task is to display a welcome message.

Print the following line exactly as shown:

```
Welcome to the SMUCT CSE FEST V.3 2026
```

Input Format

There is no input.

Constraints

Your program must print the required output exactly as specified.

Output Format

Print the following line exactly:

```
Welcome to the SMUCT CSE FEST V.3 2026
```

Sample Output 0

```
Welcome to the SMUCT CSE FEST V.3 2026
```

K-Distinct Substrings

You are given a string S consisting of lowercase English letters, and an integer K . Count the number of substrings of S that contain exactly K distinct characters. Two substrings are considered different if they occur at different (start, end) index pairs, even if the

resulting characters are identical. Count the number of pairs (i, j) with $1 \leq i \leq j \leq N$ such that the substring $S[i..j]$ contains exactly K distinct characters.

Since the answer can be large, output it modulo 1,000,000,007.

Input Format

The first line contains an integer N — the length of the string.

The second line contains a string S of length N , consisting of lowercase English letters.

The third line contains an integer K .

Constraints

- $1 \leq N \leq 10^6$
- $1 \leq K \leq 26$
- S contains only lowercase English letters (a-z).

Output Format

A single integer — the count of substrings with exactly K distinct characters, modulo 1,000,000,007.

Sample Input 0

```
5
abcba
2
```

Sample Output 0

```
5
```

Explanation 0

We need to count substrings of `abcba` that contain exactly 2 distinct characters.

Checking each substring, the ones that qualify are `ab` (indices 1 to 2), `bc` (indices 2 to 3),

bcb (indices 2 to 4), cb (indices 3 to 4), and ba (indices 4 to 5). Every other substring either has only 1 distinct character, like a, b, or cc more precisely like the single-letter substrings, or has 3 distinct characters, like abc, abcb, cba, and the full string abcba. This gives a total of 5 substrings with exactly 2 distinct characters.

Sample Input 1

```
6
aabbcc
2
```

Sample Output 1

```
8
```

Explanation 1

We need to count substrings of aabbcc that contain exactly 2 distinct characters. Checking each substring, the ones that qualify are aab, aabb, ab, and abb from the first two blocks of the string, along with bbc, bbcc, bc, and bcc from the overlap between the second and third blocks. Substrings like aa or cc don't qualify since they contain only 1 distinct character, and substrings like aabbc or the full string aabbcc don't qualify either since they contain 3 distinct characters. This gives a total of 8 substrings with exactly 2 distinct characters.

The Treasure Caravan

A caravan travels through checkpoints containing treasure. There are N checkpoints numbered from 1 to N . The i -th checkpoint contains A_i units of treasure, which may be negative. The caravan starts at checkpoint 1 and must reach checkpoint N . From checkpoint i , it can move to: $i + 1$, $i + 2$, $i + K$ where K is the given integer. Determine the maximum total treasure that can be collected upon reaching checkpoint N .

Input Format

The first line contains an integer T — the number of test cases.

For each test case:

The first line contains two integers N, K .

The second line contains N integers $A_1, A_2, A_3, \dots, A_n$.

Constraints

- $1 \leq T \leq 10$
- $2 \leq N \leq 200000$
- $2 \leq K \leq N$
- $-10^9 \leq A_i \leq 10^9$

Output Format

For each test case, print one integer - the maximum treasure obtainable.

Sample Input 0

```
3
8 4
6 -2 9 -7 2 -1 8 -4
5 3
10 -20 5 -20 15
12 5
-19 -3 6 5 23 -19 -27 13 3 22 -30
```

Sample Output 0

```
21
30
39
```

Explanation 0

For the first test case ($N = 8, K = 4$), the optimal path is $1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 8$, using moves $+2, +2, +2, +1$. This collects $A_1 + A_3 + A_5 + A_7 + A_8 = 6 + 9 + 2 + 8 + (-4) = 21$, skipping the negative checkpoints at positions 2, 4, and 6 entirely.

For the second test case ($N = 5, K = 3$), the optimal path is $1 \rightarrow 3 \rightarrow 5$, using the $+2$ move twice. This collects $A_1 + A_3 + A_5 = 10 + 5 + 15 = 30$, avoiding both checkpoints with value -20 at positions 2 and 4.

For the third test case ($N = 12, K = 5$), the optimal path is $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 10 \rightarrow 12$, using moves $+2, +1, +1, +K(=5), +2$. This collects $A_1 + A_3 + A_4 + A_5 + A_{10} + A_{12} = -19 + 6 + 5 + 23 + 22 + 2 = 39$. Checkpoint 1 is negative but unavoidable, since the caravan must start there; the rest of the path picks up the largest available values along the way, including a $+K$ jump directly from checkpoint 5 to checkpoint 10 that skips the negative stretch at checkpoints 6 and 7.

Sample Input 1

```
1
5 5
1 -100 2 -100 5
```

Sample Output 1

```
6
```

Explanation 1

$N=5, K=5$. The $+K$ jump requires $i + 5 \leq 5$ ($i \leq 0$) which is never valid — so effectively only $+1$ and $+2$ moves exist. The best path avoiding the -100 checkpoints is $1 \rightarrow 3 \rightarrow 5$: $A_1 + A_3 + A_5 = 1 + 2 + 3 = 6$.

The Number Pyramid

Deep inside the Pyramid of Numbers, there is a magical staircase. Legend says that each step reveals the natural numbers in order, starting from 1. The first step shows only the number 1. Each new step adds the next number while keeping all the previous ones.

Your task is to recreate this magical staircase.

Given an integer N, print the first N steps of the Number Staircase.

Each step i should contain all integers from 1 to i, separated by a single space.

Input Format

The first and only line contains a single integer N.

Constraints

- $1 \leq i \leq N$
- $1 \leq N < 100$

Output Format

Print the required Number Staircase.

Sample Input 0

3

Sample Output 0

```
1
1 2
1 2 3
```

Explanation 0

The staircase has 3 steps:

Step 1 displays 1.

Step 2 displays 1 2.

Step 3 displays 1 2 3.

Restoring the Oath

In the ancient city of Veyrun, nine towers stand along the Avenue of Houses, one for each founding family, built in the order their oaths were sworn. Tradition demands that each tower stand strictly taller than the one before it, a visible record that every generation built higher than the last. But an earthquake has struck. Some towers sank, others didn't move at all, and the ancient order has been broken — a tower may now stand shorter than, or equal to, the one before it. You are the city's mason. You may raise any tower's height by one brick with a single unit of labor, towers can only be built up, never torn down, since cutting stone would erase a family's history. Restore the order so that every tower is strictly taller than the tower before it, using the minimum total number of bricks. Given the current heights of the N towers, find the minimum number of bricks needed to restore the tradition.

Input Format

The first line contains an integer N — the number of elements in the array.

The second line contains N space-separated integers $A_1, A_2, \dots, A_N$.

Constraints

- $1 \leq N \leq 3 \times 10^5$
- $-10^9 \leq A_i \leq 10^9$

Output Format

A single integer — the minimum number of increment operations required.

Sample Input 0

7
1 1 1 1 1 1 1

Sample Output 0

21

Explanation 0

The array must become at least 1, 2, 3, 4, 5, 6, 7. Total increments needed = $0 + 1 + 2 + 3 + 4 + 5 + 6 = 21$.

Sample Input 1

5
4 1 3 2 5

Sample Output 1

15

Explanation 1

The array must become strictly increasing: starting from 4, 1, 3, 2, 5, each tower is raised only as much as needed to exceed the tower before it, giving 4, 5, 6, 7, 8. Total increments needed = $0 + 4 + 3 + 5 + 3 = 15$.

Harmonics of the Lexicon

A choir of M strings, each consisting only of lowercase English letters, is given to you — one string per line. For any string s , define its signature vector $f(s) = (f_1(s), f_2(s), \dots, f_{26}(s))$, where $f_c(s)$ is the number of times the c -th letter of the alphabet occurs in s .

For two strings s_i and s_j , define their resonance as

$$R(s_i, s_j) = \sum_{c=1}^{26} f_c(s_i) \cdot f_c(s_j)$$

Your task is to compute the total resonance across every unordered pair of strings in the choir:

$$\text{Answer} = \sum_{1 \leq i < j \leq M} R(s_i, s_j)$$

Since this value can be very large, output it modulo $10^9 + 7$.

Function Description

Complete the function `sumOfResonance` in the editor below.

`sumOfResonance` has the following parameter(s):

- `string s[M]`: an array of M strings, one per line, each consisting of lowercase English letters

Returns

int: the value $(\sum_{i < j} R(s_i, s_j)) \bmod (10^9 + 7)$

Input Format

The first line contains a single integer T , the number of test cases.

For each test case: - The first line contains a single integer M , the number of strings in the choir. - Each of the next M lines contains a single string s_i .

Constraints

- $1 \leq T \leq 20$
- $1 \leq M \leq 10^5$

- $1 \leq |s_i| \leq 10^5$ for every i
- The sum of $|s_i|$ over all strings within a single test case does not exceed 2×10^5
- Every s_i consists only of lowercase English letters (a to z).

Output Format

For each test case, output a single line containing the answer: the total resonance across all pairs, modulo $10^9 + 7$.

Sample Input 0

```
1
3
aab
abc
bcc
```

Sample Output 0

```
7
```

Explanation 0

The signature vectors (restricted to letters a, b, c, since no other letters appear) are:
 $f(aab) = (2, 1, 0)$, $f(abc) = (1, 1, 1)$, $f(bcc) = (0, 1, 2)$

Computing resonance for each pair:

$$R(aab, abc) = 2 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$$

$$R(aab, bcc) = 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 2 = 1$$

$$R(abc, bcc) = 1 \cdot 0 + 1 \cdot 1 + 1 \cdot 2 = 3$$

$$\text{Total} = 3 + 1 + 3 = 7.$$

Sample Input 1

```
3
3
aab
abc
bcc
1
zzzzz
4
abc
abc
abc
abc
```

Sample Output 1

```
7
0
18
```

Explanation 1

Scenario 1 — $M = 3$ strings: aab, abc, bcc. Same as the original sample: pairwise resonances are $R(\text{aab}, \text{abc}) = 3$, $R(\text{aab}, \text{bcc}) = 1$, $R(\text{abc}, \text{bcc}) = 3$, summing to 7.

Scenario 2 — $M = 1$, just zzzzz. With only one string in the choir, there are no pairs (i, j) with $i < j$ at all, so the sum is vacuously 0 — regardless of how long or repetitive the single string is. This checks that the solution correctly handles the boundary $M = 1$ (the minimum allowed by the constraints) rather than crashing or miscounting on an empty pair set.

Scenario 3 — $M = 4$ identical strings, all abc. Every string has signature vector $f = (1, 1, 1, 0, \dots, 0)$ (one each of a, b, c). Since all four are identical, every pair resonates identically too: $R(s_i, s_j) = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 3$ for every pair. There are $(4 \text{ choose } 2) = 6$ pairs, giving total $6 \times 3 = 18$. This is a good check that the $O(M)$ frequency-based formula agrees with the naive pairwise count when many duplicates are present — a case where a buggy implementation (say, one that only counts pairs of identical strings via a hash map

instead of summing resonance across every pair) would silently diverge from the correct value.

The Guildmaster's Selection

The guildmaster of an ancient order must assemble a team by choosing r apprentices out of a pool of n candidates, where the order of selection does not matter. For each of several missions, you are given a pair (n, r) and must determine the number of distinct ways the guildmaster can form the team that is, compute the binomial coefficient

$$\binom{n}{r} = n! / (r! (n-r)!)$$

Since this value can be extremely large, output it modulo $10^9 + 7$.

Function Description

Complete the function `countSelections` in the editor below.
`countSelections` has the following parameter(s):

- `int n`: the number of candidates
- `int r`: the number of apprentices to be chosen

Returns

- `int`: the value $\binom{n}{r} \bmod (10^9 + 7)$

Input Format

The first line contains a single integer T , the number of missions (test cases).
Each of the next T lines contains two space-separated integers n and r , describing one mission.

Constraints

- $1 \leq T \leq 10^5$

- $0 \leq r \leq n \leq 10^6$

Output Format

For each mission, output a single line containing $(n \text{ choose } r) \bmod (10^9 + 7)$.

Sample Input 0

```
3
5 2
6 0
10 10
```

Sample Output 0

```
10
1
1
```

Explanation 0

For the first mission, $(5 \text{ choose } 2) = 5! / (2! 3!) = 10$: choosing 2 apprentices out of 5 candidates can be done in 10 distinct ways.

For the second mission, $(6 \text{ choose } 0) = 1$: there is exactly one way to choose zero apprentices so choose none at all.

For the third mission, $(10 \text{ choose } 10) = 1$: there is exactly one way to choose all 10 candidates so take everyone.

Sample Input 1

```
3
0 0
7 3
1000000 500000
```

Sample Output 1

```
1
35
996692777
```

Explanation 1

In the first test case, $n = 0$, $r = 0$. $(0 \text{ choose } 0) = 1$: there is exactly one way to choose zero apprentices from a pool of zero candidates — select nothing. This checks the lower boundary of the constraints ($n = 0$ is the smallest allowed value), where the empty product convention ($0! = 1$) must be handled correctly rather than treated as an error case.

In the second test case, $n = 7$, $r = 3$. $(7 \text{ choose } 3) = 7! / (3! 4!) = 5040 / (6 \times 24) = 35$: a straightforward mid-sized case with no modular wraparound, useful for confirming the factorial/inverse-factorial precomputation gives the exact textbook value before modular arithmetic effects come into play.

In the third test case, $n = 1000000$, $r = 500000$. This pushes n to the constraint's upper bound 10^6 with r at the midpoint, where $(n \text{ choose } r)$ is astronomically large in ordinary arithmetic — far too big to compute directly, which is exactly why the modular precomputation approach is required. Under modulo $10^9 + 7$, the answer comes out to 996692777. This case specifically stresses that `fact[]` and `invfact[]` are precomputed up to the full 10^6 bound (an implementation that only precomputes up to, say, 10^5 by mistake would fail here) and that the modular inverse exponentiation was done correctly rather than via a slow per-query inverse.

Paths of Least Peril

A trade guild operates across n cities, connected by m bidirectional roads. Each road i connects cities u_i and v_i and carries a risk level w_i — the danger a caravan faces while traveling on it.

For a path P between two cities, define its peril as the maximum risk level among the roads used on that path:

$$\text{peril}(P) = \max_{e \in P} w_e$$

For each of q trade queries, given a pair of cities (a, b) , the guild wants to know the minimum possible peril over all paths from a to b — that is,

$$\text{answer}(a, b) = \min_{P: a \rightarrow b} \text{peril}(P)$$

If a and b lie in different connected components (no path exists at all), report -1 for that query.

Function Description

Complete the function `minimizeMaxRisk` in the editor below.

`minimizeMaxRisk` has the following parameter(s):

- `int n`: the number of cities, labeled 1 to n
- `int u[m], v[m], w[m]`: the endpoints and risk level of each of the m roads
- `int a[q], b[q]`: the two endpoint cities of each of the q queries

Returns

`int ans[q]`: for each query i , the value `answer(a_i, b_i)` as defined above, or -1 if a_i and b_i are disconnected

Input Format

The first line contains a single integer T , the number of test cases.

For each test case:

The first line contains two space-separated integers n and m , the number of cities and roads. Each of the next m lines contains three space-separated integers u_i, v_i, w_i , describing one road.

The next line contains a single integer q , the number of queries.

Each of the next q lines contains two space-separated integers a_j and b_j , describing one query.

Constraints

- $1 \leq T \leq 10$
- $1 \leq n \leq 10^5$
- $0 \leq m \leq 2 \times 10^5$
- $1 \leq u_i, v_i \leq n, u_i \neq v_i$
- $1 \leq w_i \leq 10^9$
- $1 \leq q \leq 10^5$
- $1 \leq a_j, b_j \leq n$
- The graph may be disconnected, and may contain multiple roads between the same pair of cities.

Output Format

For each query, output a single line containing $\text{answer}(a_j, b_j)$, or -1 if no path exists.

Sample Input 0

```
1
6 5
4 5 1
2 3 2
1 3 3
1 2 4
3 4 5
3
1 5
2 4
1 6
```

Sample Output 0

```
5
5
-1
```

Explanation 0

Cities 1 through 5 are connected by the 5 given roads; city 6 is isolated.

Sorting the roads by risk level: (4, 5, 1), (2, 3, 2), (1, 3, 3), (1, 2, 4), (3, 4, 5). Running Kruskal's algorithm (adding an edge only if it connects two different components) selects: (4, 5, 1), (2, 3, 2), (1, 3, 3), and (3, 4, 5) — the road (1, 2, 4) is skipped because 1 and 2 are already in the same component once (1, 3, 3) and (2, 3, 2) have been added, so adding it would form a cycle.

This gives the tree 4 - 5, 2 - 3, 1 - 3, 3 - 4, i.e. city 3 is the hub connecting to 1, 2, and 4, with 4 further connected to 5.

For query (1, 5): the only tree path is $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$, using risk levels 3, 5, 1. The peril is $\max(3, 5, 1) = 5$.

For query (2, 4): the tree path is $2 \rightarrow 3 \rightarrow 4$, using risk levels 2, 5. The peril is $\max(2, 5) = 5$.

For query (1, 6): city 6 has no roads at all, so it is disconnected from city 1. The answer is -1.

Sample Input 1

```

2
6 5
4 5 1
2 3 2
1 3 3
1 2 4
3 4 5
3
1 5
2 4
1 6
4 4
1 2 2
2 3 1
1 3 100
2 3 1
3
1 3
2 1
1 4

```

Sample Output 1

```

5
5
-1
2
2
-1

```

Explanation 1

Case 1: is the original sample, cities 1-5 connected by 5 roads, city 6 isolated. The MST selects roads (4, 5, 1), (2, 3, 2), (1, 3, 3), (3, 4, 5), skipping (1, 2, 4) since it would form a cycle. Query (1, 5) walks $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ with peril $\max(3, 5, 1) = 5$; query (2, 4) walks $2 \rightarrow$

3 → 4 with peril $\max(2, 5) = 5$; query (1, 6) is disconnected, giving -1.

Case 2: tests parallel edges: cities 1, 2 are connected by two roads one with risk 2 and one with risk 10 plus a road (1, 3, 100) and (2, 3, 1), with city 4 left isolated. Sorting by weight: (2, 3, 1), (1, 2, 2), (1, 2, 10), (1, 3, 100). Kruskal adds (2, 3, 1) and (1, 2, 2), then skips both remaining roads since 1, 2, 3 are already fully connected including correctly ignoring the heavier duplicate (1, 2, 10) rather than mistakenly using it. This is the key thing the test checks: an implementation that doesn't dedupe/compare parallel edges by weight during Kruskal (one that just picks the first edge seen between a pair rather than the cheapest) would build the wrong tree here. With the correct MST, path 1 → 2 → 3, query (1, 3) gives peril $\max(2, 1) = 2$; query (2, 1) uses the direct MST edge, peril 2; query (1, 4) hits the isolated node, giving -1.

Hackerman's Network Sweep

Hackerman's network consists of r routers connected in a full mesh (every pair of routers is directly connected), and $n - r$ devices, each belonging to exactly one router's local cluster. Within a cluster, devices may be connected to their router and/or to each other, but never to a device or router outside their own cluster — except for exactly one rogue edge, which connects two nodes belonging to different clusters.

Given the scanned network, find the rogue edge, compute shortest distances from the start node with and without it, and answer a bandwidth-constrained shortest path query.

It is guaranteed that removing exactly one edge from the graph restores a valid mesh-and-clusters topology, and that the graph is connected.

Function Description

Complete the function `networkSweep` in the editor below.

`networkSweep` has the following parameter(s):

- `int n`: the number of nodes
- `int[] routers`: 1-indexed labels of the router nodes

- `int[][] edges`: each row is `[u, v, w, b]` — an undirected edge between `u` and `v` with latency `w` and bandwidth `b`
- `int s`: the start node
- `int t`: the target node
- `int bMin`: minimum required bandwidth for the constrained query

Returns

A Result object (or equivalent struct/dict in your language) with:

- `int[] rogueEdge`: the two endpoints of the rogue edge, smaller label first
- `long[] distFull`: shortest distance from `s` to every node `1..n`, using the full graph
- `long[] distWithoutRogue`: same, with the rogue edge removed
- `long constrainedDist`: shortest latency from `s` to `t` using only edges with bandwidth $\geq bMin$, or `-1` if unreachable

Input Format

The first line contains two space-separated integers `n` and `r`.

The second line contains `r` space-separated integers, the router labels.

The third line contains an integer `m`, the number of edges.

Each of the next `m` lines contains four space-separated integers `u v w b`.

The last line contains three space-separated integers `s t bMin`.

Constraints

- $3 \leq r \leq n \leq 2000$
- $n - 1 \leq m \leq 4000$
- $1 \leq u_i, v_i \leq n, u_i \neq v_i$
- $1 \leq w_i, b_i \leq 10^4$
- $1 \leq s, t \leq n$
- $1 \leq bMin \leq 10^4$
- No multi-edges. The graph is undirected and connected.
- Exactly one rogue edge is guaranteed to exist, and it is guaranteed to be uniquely identifiable (removing any other single edge never restores a valid topology).

Output Format

Print four lines (handled by the provided driver code — you only implement `networkSweep`):

1. The rogue edge's two endpoints, space-separated, smaller first.
2. n space-separated integers: `distFull`.
3. n space-separated integers: `distWithoutRogue`.
4. `constrainedDist`.

Sample Input 0

```
12 4
1 2 3 4
18
1 2 4 50
1 3 6 50
1 4 9 50
2 3 3 50
2 4 7 50
3 4 3 50
3 5 2 20
1 6 5 20
1 7 8 20
5 6 2 20
5 7 3 20
2 8 2 20
3 9 3 20
3 10 6 20
4 11 2 20
4 12 4 20
11 12 2 20
7 11 1 10
1 12 15
```

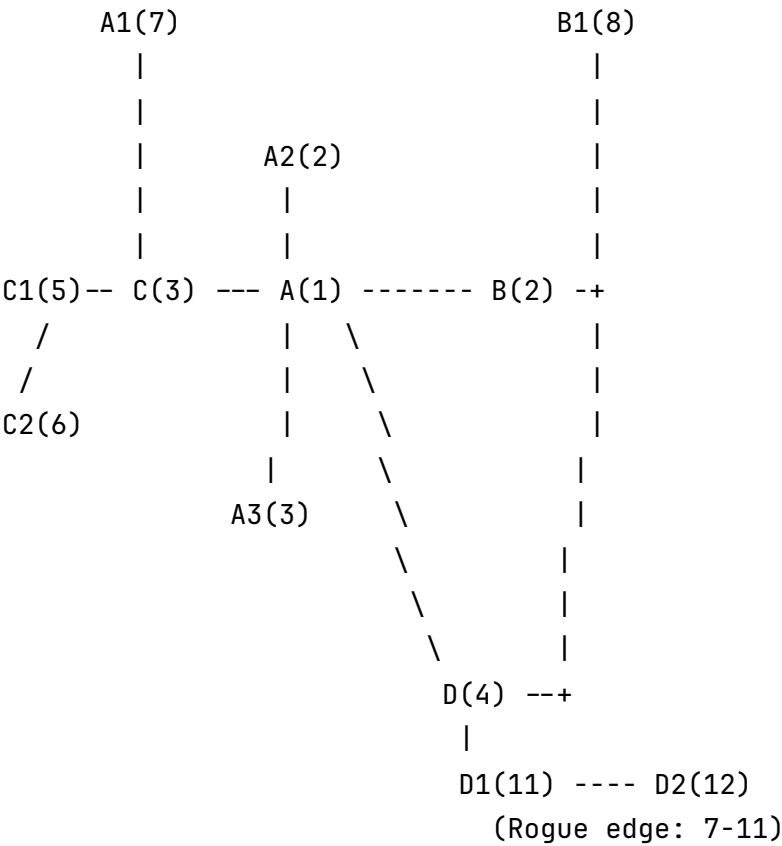
Sample Output 0

7 11
0 4 6 9 2 4 7 6 9 12 8 10
0 4 6 9 2 4 7 6 9 12 11 13
13

Explanation 0

Sample Input 0 — Network graph (numbers = edge latency)

Graph for Sample Input 0 – Network graph (numbers = edge latency)



Legend: Router (square), Device (circle), Rogue edge (dashed)

Node 7 and node 11 belong to different clusters — 7 is in router 1's cluster, 11 is in router 4's cluster — and edge 7-11 is the only edge in the graph that crosses clusters without passing through a router. Removing it restores a valid mesh-and-clusters

topology, so 7-11 is the rogue edge. With the rogue edge present, the shortest distance from node 1 to node 11 is 8 (path 1-5-6-7-11, cost $2+2+3+1$). Removing it, the shortest path becomes 1-4-11 (cost $9+2 = 11$). For the bandwidth query, edge 7-11 has bandwidth 10, below the required 15, so it cannot be used. The shortest path from node 1 to node 12 using only edges with bandwidth ≥ 15 is 1-4-12, with latency $9 + 4 = 13$.